

Fondasi Projek

Saat mempelajari diagram untuk development ambil dari diagram yang ada di draw io
<https://app.diagrams.net/#G1dc8MpgRVtE664CPTQIAFVhH8AxOQEzjz#%7B%22pageId%22%3A%22bHNOJM82YqE8PVNOBiBo%22%7D>

A. Install WSL

```
wsl --install
```

B. Install Docker di WSL

```
# Update sistem dasar
```

```
sudo apt-get update
```

```
sudo apt-get install ca-certificates curl
```

```
# Buat folder keyring dan unduh GPG key Docker
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o  
/etc/apt/keyrings/docker.asc
```

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
# Tambahkan repositori Docker ke source list Ubuntu
```

```
echo \
```

```
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
```

```
https://download.docker.com/linux/ubuntu \
```

```
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
```

```
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
sudo apt-get update
```

```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
```

```
docker-compose-plugin -y
```

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

```
sudo service docker start
```

```
sudo systemctl enable docker.service
```

```
sudo systemctl enable containerd.service
```

C. Buat Compose di docker

```
# Buat folder di File Explorer untuk tempat minio upload
```

```
# Contoh D:\SMT 6\SAD\staging22\data
```

```
- "/mnt/d/SMT 6/SAD/staging22/data:/data"
```

```
mkdir project-sad22
```

```
cd project-sad22
```

```
nano docker-compose.yml
```

```
version: '3.8'
```

```
services:
```

```
# 1. Database Metadata
```

Saat mempelajari diagram untuk development ambil dari diagram yang ada di draw io
<https://app.diagrams.net/#G1dc8MpgRVtE664CPTQIAFVhH8AxOQEzjz#%7B%22pageId%22%3A%22bHNOJM82YqE8PVNOBiBo%22%7D>

```
postgres_db:
  image: postgres:15
  container_name: sad_postgres
  restart: unless-stopped
  environment:
    POSTGRES_USER: admin_sad
    POSTGRES_PASSWORD: password_db
    POSTGRES_DB: arsip_db
  ports:
    - "2432:5432"
  volumes:
    - ./data/postgres:/var/lib/postgresql/data
```

2. Storage File Fisik (S3 Compatible via NFS)

```
minio_storage:
  image: minio/minio
  container_name: sad-minio
  restart: unless-stopped
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: admin_minio
    MINIO_ROOT_PASSWORD: password_minio
  ports:
    - "2000:9000" # Port API
    - "2001:9001" # Port Web Console
  volumes:
```

```
  # Arahkan langsung ke mount TrueNAS Anda
#   - /mnt/staging_sad22:/data
```

```
  - "/mnt/d/SMT6/SAD/staging22/data:/data"
```

3. Environment Machine Learning & EDA

```
jupyter_lab:
  image: jupyter/scipy-notebook:latest
  container_name: sad_jupyter
  restart: unless-stopped
  environment:
    JUPYTER_ENABLE_LAB: "yes"
  ports:
    - "2888:8888"
  volumes:
    - ./notebooks:/home/jovyan/work
    - ./ml_model:/home/jovyan/ml_model
  command: start-notebook.sh --NotebookApp.token="" # Akses tanpa password
```

4. Database Management (Web GUI)

```
cloudbeaver:
  image: dbeaver/cloudbeaver:latest
```

Saat mempelajari diagram untuk development ambil dari diagram yang ada di draw io
<https://app.diagrams.net/#G1dc8MpgRVtE664CPTQIAFVhH8AxOQEzjz#%7B%22pageId%22%3A%22bHNOJM82YqE8PVNOBiBo%22%7D>

```
container_name: sad_cloudbeaver
restart: unless-stopped
ports:
  - "2978:8978"
volumes:
  - ./data/cloudbeaver:/opt/cloudbeaver/workspace
depends_on:
  - postgres_db
```

```
networks:
  default:
    external: true
    name: sad_net
```

untuk menyimpan tekan CTRL X lalu Y setelah itu enter
docker compose up -d

D. Persiapan dan Clone Repo Private

```
mkdir -p ~/.ssh
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -C "dell-wsl-aziz"
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
# masukan ssh key yang sudah dibuat di setting github (profile>setting>Access>SSH
keys>New SSH Key Authentication)
nano ~/.bashrc
```

```
if [ -z "$SSH_AUTH_SOCK" ]; then
  eval "$(ssh-agent -s)" > /dev/null
  ssh-add ~/.ssh/id_ed25519 2>/dev/null
fi
```

```
source ~/.bashrc
ssh -T git@github.com
# Hasilnya harus seperti ini Hi UmarAziz01! You've successfully authenticated, but
GitHub does not provide shell access.
git clone git@github.com:UmarAziz01/arsip-sad-app.git
```

E. Masuk ke Folder & Berpindah Branch

```
cd arsip-sad-app
git fetch origin
```

```
# Jalankan perintah sesuai nama masing-masing:
git checkout feat/ai-aulia # (Untuk Aulia)
git checkout feat/data-laura # (Untuk Laura)
```

Saat mempelajari diagram untuk development ambil dari diagram yang ada di draw io
<https://app.diagrams.net/#G1dc8MpgRVtE664CPTQIAFVhH8AxOQEzjz#%7B%22pageId%22%3A%22bHNOJM82YgE8PVNOBiBo%22%7D>

git checkout feat/infra-aziz # (Untuk Laura)

F. Setup Environment (Instalasi Library)

>> Backend (FastAPI)

cd backend

sudo apt install python3.12-venv

python3 --version

python3 -m venv venv

source venv/bin/activate

pip install -r requirements.txt

>> Frontend (Vue)

Masuk ke direktori frontend

cd frontend

install node nvm dll

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
export NVM_DIR="$([ -z "${XDG_CONFIG_HOME-}" ] && echo "$HOME/.nvm" ||
echo "$XDG_CONFIG_HOME/nvm")"
```

```
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
```

tutup terminal dan buka lagi

nvm install 22.13.1

setelah selesai

nvm use 22.13.1

nvm alias default 22.13.1

npm install -g npm@11.3.0

node -v # Harus v22.13.1

npm -v # Harus 11.3.0

Menginstal semua dependensi (Vue, Vite, Tailwind, dll)

sudo apt install npm

npm install

(Opsional) Jika ingin memastikan format kode rapi sesuai aturan Aziz

npm run format

Menjalankan server development Vue

npm run dev

INSTALL LIBRARY VSCODE

1. Vue - Official (Volar)

2. Tailwind CSS IntelliSense

3. ESLint & Prettier

G. Konfigurasi Environment File (.env)

Di folder backend

cp .env.example .env

Saat mempelajari diagram untuk development ambil dari diagram yang ada di draw io
<https://app.diagrams.net/#G1dc8MpgRVtE664CPTQIAFVhH8AxOQEzjz#%7B%22pageId%22%3A%22bHNOJM82YgE8PVNOBiBo%22%7D>

```
# Di folder frontend  
cp .env.example .env
```

```
# Lalu isi nilai di dalam file .env sesuai dengan konfigurasi server Tailscale/Local  
yang diberikan Aziz
```

H. Menjalankan Aplikasi (Backend)

```
cd backend  
source venv/bin/activate  
uvicorn main:app --reload --host 0.0.0.0 --port 8000  
#nohup venv/bin/uvicorn main:app --host 0.0.0.0 --port 8000 > log.txt 2>&1 &
```

I. Menjalankan Aplikasi (Frontend)

```
cd frontend  
npm run dev
```

J. Konfig MinIO

Buka web MinIO <http://localhost:2001/browser/>
login sesuai dengan docker compose
username admin_minio
password password_minio
Buat bucket dengan nama “arsip-dokumen”

K. Konfig Tambahan

```
sudo chown -R ubuntu:ubuntu /home/ubuntu/project-sad22/ml_model  
chmod -R 755 /home/ubuntu/project-sad22/ml_model  
sudo chmod -R 777 ./data/cloudbeaver
```

Git Aziz

<https://gemini.google.com/u/8/app/391b2190a49a30dd>

<https://gemini.google.com/u/8/app/c423ae8eeed8c2bc>

<https://gemini.google.com/app/7e4d9af09f1d8854>

<https://gemini.google.com/u/1/app/ee3bd737c73bea46> CI CD

<https://github.com/hsiangfeng/example-vue3-vitest/blob/main/package.json>

<https://github.com/kirklin/skills/blob/main/skills/vue-testing-best-practices/reference/testing-vitest-recommended-for-vue.md>



Dokumen 1: Panduan Peran Aziz (Project Lead)

Tanggung Jawab: Menjaga kebersihan `main` & `develop`, melakukan Code Review, dan melakukan Merge.

- **Tugas Rutin: Melakukan Merge Pull Request**
 1. Buka tab **Pull Requests** di GitHub.
 2. Pilih PR dari Aulia atau Laura.
 3. Lihat bagian "Files Changed" untuk memastikan tidak ada kode yang aneh.
 4. Jika oke, klik **Merge Pull Request**.

Tugas Insidental: Rilis ke Main (Demo/Sidang) Jika kode di `develop` sudah stabil dan siap demo:

Bash

git checkout main

git merge develop

git push origin main

- **Tips Lead:** Jika ada konflik yang tidak bisa diselesaikan anggota, bantu mereka melakukan *manual merge* di komputer kamu.



Dokumen 2: Panduan Peran Aziz (Infra Member)

Branch Utama: `feat/infra-aziz` | **Fokus:** Setup Database, FastAPI, & Infrastruktur.

Siklus Kerja Harian:

Bash

1. Pastikan di branch sendiri

git checkout feat/infra-aziz

2. Ambil update dari teman (via develop)

git fetch origin

git merge origin/develop

3. Koding... (Setup models.py, dll)

4. Simpan & Push

git add .

git commit -m "feat: setup models database v2"

git push origin feat/infra-aziz

- **Setelah Push:** Buka GitHub dan buat Pull Request ke branch **develop**.
- Push sesering mungkin ke branch fitur masing-masing tapi pastikan tidak ada error, test dulu jalan atau tidak baru lakukan commit
- Lakukan Pull Request (PR) setelah menyelesaikan satu fungsi (misal: fungsi `upload_file`).

Tips untuk Semua (Golden Rules):

1. **Jangan Pernah Koding di Branch **develop** atau **main**.** Selalu di branch **feat/** masing-masing.
2. **Pull Sebelum Push.** Selalu lakukan **git merge origin/develop** sebelum kamu melakukan **push** agar kamu tahu jika ada kode teman yang konflik dengan kodemu.
3. **Jenkins Ready:** Karena ke depannya akan ada Jenkins, mulai sekarang biasakan **pesan commit yang jelas**. Jenkins biasanya akan membaca pesan commit ini untuk membuat laporan otomatis.

1. Membuat Sesi Baru dengan Nama

Untuk memulai sesi dengan nama khusus, jalankan perintah berikut di terminal:

- `tmux new -s nama_sesi`

Contoh: Jika Anda ingin membuat sesi bernama "project1", ketik `tmux new -s project1`.

3. Mengelola Banyak Sesi

Setelah memiliki beberapa sesi, Anda bisa menggunakan perintah berikut untuk berpindah atau melihat daftar sesi:

- Melihat daftar sesi: Gunakan perintah `tmux ls` atau `tmux list-sessions` untuk melihat semua sesi yang sedang aktif.
- Masuk kembali ke sesi tertentu: Gunakan perintah `tmux attach -t nama_sesi`.
- Berpindah antar sesi (saat di dalam tmux): Tekan `Ctrl+b` lalu `s` untuk memilih sesi dari daftar interaktif.
- Mengubah nama sesi (saat di dalam tmux): Tekan `Ctrl+b` lalu `$` untuk mengganti nama sesi yang sedang berjalan.

Untuk bisa melakukan scroll atau menyeret layar ke atas dan ke bawah di tmux, Anda memiliki dua opsi utama: menggunakan keyboard (Mode Salin) atau mengaktifkan dukungan mouse.

1. Menggunakan Keyboard (Copy Mode)

Ini adalah cara standar tanpa perlu mengubah konfigurasi apa pun:

- Masuk ke mode scroll: Tekan `Ctrl + b` lalu tekan tombol `[`.
- Navigasi: Gunakan tombol Panah Atas/Bawah atau `Page Up`/`Page Down` untuk melihat riwayat teks.

- Keluar: Tekan tombol q untuk kembali ke baris perintah normal.
- ## 2. Mengaktifkan Mouse (Scroll dengan Roda Mouse)
- Jika Anda ingin bisa langsung menggunakan scroll wheel pada mouse seperti terminal biasa, Anda harus mengaktifkan fiturnya terlebih dahulu:
- Cara Sementara (Hanya sesi ini):
 1. Tekan Ctrl + b lalu tekan : untuk masuk ke mode perintah.
 2. Ketik set -g mouse on dan tekan Enter.
 - Cara Permanen (Setiap kali buka tmux):
 1. Buka (atau buat) file konfigurasi tmux di direktori home Anda: nano ~/.tmux.conf.
 2. Masukkan baris berikut: set -g mouse on.
 3. Simpan dan keluar, lalu muat ulang konfigurasi dengan perintah tmux source-file ~/.tmux.conf di dalam terminal.
- Catatan: Jika Anda mengaktifkan mode mouse, menyeret teks untuk menyalin (copy) mungkin akan terasa berbeda karena tmux akan menangkap input mouse tersebut. Anda biasanya perlu menahan tombol Shift sambil menyeret mouse jika ingin menyalin teks ke clipboard sistem operasi.

Ringkasan Cepat Biar Tidak Tertukar:

Aksi	Dari (Sumber)	Ke (Tujuan)	Tombol yang Digunakan
Copy	Luar (Browser)	-	Ctrl + C
Paste	-	Tmux	Ctrl + Shift + V atau Klik Kanan
Copy	Tmux	Luar (WA/Browser)	Tahan Shift + Blok + Ctrl + Shift + C
Paste	-	Luar	Ctrl + V

Catatan Penting:

Jangan gunakan Ctrl + b lalu] untuk teks dari luar, karena itu hanya untuk menempelkan teks yang disalin di dalam sesi tmux itu sendiri (internal buffer).

Setting TMUX

1. Buka (atau buat) file konfigurasi tmux di direktori home Anda: nano ~/.tmux.conf.
2. Masukkan baris berikut: **set -g mouse on.**
3. Simpan dan keluar, lalu muat ulang konfigurasi dengan perintah **tmux source-file ~/.tmux.conf** di dalam terminal.

Jika Anda sudah mengaktifkan set -g mouse on: CARA COPY PAKAI MOUSE

1. Klik dan tahan tombol kiri mouse pada awal teks yang ingin disalin.
2. Seret (drag) mouse ke bawah (atau ke atas). Jika Anda menyeret mouse hingga melewati batas bawah jendela tmux, layar akan otomatis melakukan auto-scroll sehingga Anda bisa terus memblok teks hingga baris yang diinginkan.

3. Lepaskan klik mouse. Secara default, teks yang diblok akan otomatis tersalin ke buffer internal tmux.
4. Menempel (Paste): **Tekan Ctrl + b lalu]** untuk menempelkan teks tersebut di terminal tmux.

Jika Anda sudah mengaktifkan set -g mouse on: CARA COPY PAKAI KEYBOARD

1. Masuk ke mode salin: Tekan Ctrl + b lalu [.
2. Gerakkan kursor ke awal teks menggunakan tombol panah atau Page Up/Down.
3. Mulai seleksi: Tekan tombol Spasi (atau tombol v jika Anda menggunakan mode Vi).
4. Gunakan tombol panah bawah atau Page Down untuk menyorot banyak baris. Layar akan otomatis bergeser ke bawah saat kursor mencapai batas layar.
5. Selesaikan salinan: Tekan Enter (atau tombol y di mode Vi).
6. Menempel (Paste): Tekan Ctrl + b lalu]

Setting History Limit DI TMUX

Secara default, tmux biasanya hanya menyimpan sekitar 2.000 baris. Anda bisa meningkatkannya menjadi 10.000 atau lebih.

1. Cara Mengubah secara Permanen

Lakukan langkah-langkah ini di terminal:

1. Buka file konfigurasi:
`nano ~/.tmux.conf`
2. Tambahkan baris berikut (misal ingin 10.000 baris):
`set -g history-limit 10000`
3. Simpan dan Keluar:
Tekan Ctrl + o, lalu Enter, kemudian Ctrl + x.
4. Muat ulang konfigurasi:
Agar perubahan langsung aktif tanpa mematikan sesi, jalankan perintah ini di dalam tmux:
`tmux source-file ~/.tmux.conf`

2. Cara Mengubah secara Sementara

Jika Anda sedang berada dalam sesi dan hanya ingin mengubah batasnya untuk sesi tersebut saja (tanpa edit file):

1. Tekan Ctrl + b lalu :
2. Ketik: `set -g history-limit 5000`
3. Tekan Enter.

Perbandingan Pengaturan (Pilih Salah Satu)

Kondisi	<code>secure</code>	<code>samesite</code>	Kebutuhan
Akses via IP ke IP (HTTP)	<code>False</code>	<code>"lax"</code>	Cocok untuk lokal/
Akses via Domain (HTTPS)	<code>True</code>	<code>"none"</code>	Wajib untuk Produksi/Cloudflare

Sedikit Tips untuk Langkah Terakhir

Meskipun sudah jalan di lingkungan IP (100.100.30.51), ingat dua hal ini jika nanti kamu pindah ke domain produksi (aziz-sad.61262.online):

- Pindah ke HTTPS: Begitu pakai domain, pastikan ganti kembali ke `secure=True` dan `samesite="none"` di backend agar Chrome tidak memblokir cookie-nya lagi.
- Hapus Log Debug: Jika sudah stabil, kamu bisa menghapus `console.log` di interceptor agar konsol browser tidak terlalu ramai.

Secara bawaan (default), jawabannya adalah **TIDAK**. Branch tersebut masih akan tetap ada di repositori GitHub Anda meskipun sudah berhasil di-merge ke `develop` atau `main`.

Namun, sebagai standar industri dan *best practice* seorang Lead/DevOps, **branch fitur tersebut HARUS dihapus** setelah berhasil di-merge.

Berikut adalah alasan dan cara menanganinya:

Kenapa harus dihapus?

1. **Menjaga Kerapian (Clean Repo):** Jika Anda punya 5 anggota tim dan masing-masing membuat 4 fitur sebulan, dalam setahun Anda akan punya ratusan *branch* mati yang menumpuk. Ini akan sangat membingungkan.
2. **Menyegarkan Jenkins:** Ingat *branch* Anda yang dicoret (Disabled) di Jenkins tadi? Jika *branch* tersebut Anda hapus dari GitHub, maka di Jenkins juga akan otomatis terhapus dengan bersih, membuat *dashboard* CI/CD Anda rapi kembali.

Cara Menghapusnya (3 Cara)

1. Cara Manual di GitHub (Paling Umum) Sesaat setelah Anda menekan tombol hijau "Merge pull request" di GitHub, biasanya akan langsung muncul tombol abu-abu/ungu bertuliskan "**Delete branch**". Anda cukup menekan tombol tersebut.

2. Cara Otomatis (Rekomendasi untuk Lead) Daripada Anda atau tim harus ingat untuk menekan tombol "Delete branch" setiap saat, Anda bisa menyuruh GitHub melakukannya secara otomatis:

- Buka repositori GitHub Anda.
- Pergi ke tab **Settings** (Pengaturan).
- Di menu *General*, gulir ke bawah ke bagian *Pull Requests*.
- Centang kotak "**Automatically delete head branches**".
- Mulai sekarang, setiap ada PR yang di-merge, *branch* sumbernya akan otomatis musnah!

3. Jangan Lupa Bersihkan di Komputer Lokal (WSL) Anda! Menghapus *branch* di GitHub **tidak** otomatis menghapusnya di komputer lokal Anda. Untuk membersihkannya di terminal WSL Anda, jalankan dua perintah ini:

Bash

```
# Pindah dulu ke develop agar Anda tidak sedang 'menduduki' branch yang mau dihapus
git checkout develop
```

Hapus branch fitur di komputer lokal Anda
git branch -d feat/infra-aziz

Bersihkan "jejak hantu" branch yang sudah dihapus di GitHub agar lokal Anda ikut sinkron
git fetch --prune

Dengan menerapkan siklus ini (Buat Branch -> Commit -> Push -> PR -> Merge -> Hapus Branch), repositori Arsip Pintar Anda akan selalu bersih, profesional, dan setara dengan standar perusahaan teknologi besar!

feat/... : Hanya digunakan saat membuat fitur yang benar-benar **baru**. (Contoh: **feat/delete-arsip**)

fix/... : Digunakan saat ada **error/bug** di fitur yang sudah dirilis. (Contoh: **fix/button-delete-hilang**)

enhancement/... atau **update/...** : Digunakan saat fitur lama berjalan normal, tapi Anda ingin **meningkatkan** kinerjanya atau memperbarui tampilannya. (Contoh: **enhancement/animasi-delete**)

ENV Backend

```
ALLOWED_ORIGINS=https://aziz-sad.61262.online,http://localhost:5173,http://localhost:8000,http://100.100.30.51:5173,

# DATABASE (Sesuaikan dengan database.py kamu)
DB_USER=admin_sad
DB_PASSWORD=password_db
DB_HOST=localhost
DB_PORT=2432
DB_NAME=arsip_db

# JWT SETTINGS
SECRET_KEY=rahasia-super-kuat-archiflow-ganti-nanti
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=10
REFRESH_TOKEN_EXPIRE_DAYS=1

# MINIO SETTINGS
MINIO_URL=localhost:2000
MINIO_ACCESS_KEY=admin_minio
MINIO_SECRET_KEY=password_minio
BUCKET_NAME=arsip-dokumen

# SMTP SETTINGS (Gmail)
SMTP_SERVER=smtp.gmail.com
SMTP_PORT=587
SENDER_EMAIL=labsifst@gmail.com
SENDER_PASSWORD=utux kxhu kzpo ksgx
FRONTEND_PROTOCOL=https
FRONTEND_HOST=localhost
FRONTEND_PORT=5173
FRONTEND_RESET_PATH=/reset-password
```

ENV Frontend


```
VITE_API_BASE_URL=http://localhost:8000
VITE_APP_NAME="Arsip Pintar (env)"
VITE_APP_MOTTO="Enterprise Archive Motto (env)"
VITE_APP_DESCRIPTION="AI-driven document classification and archival
(env)"
# VITE_MINIO_CONSOLE=http://10.0.2.129:2001
```

Git Aulia

Dokumen 3: Panduan Peran Aulia (AI Specialist)

Branch Utama: `feat/ai-aulia` | **Fokus:** Integrasi OCR & Logika AI.

Siklus Kerja Harian:

Bash

1. Masuk ke branch AI

`git checkout feat/ai-aulia`

2. Sinkronisasi agar kode tidak bentrok dengan Aziz/Laura

`git fetch origin`

`git merge origin/develop`

3. Koding... (Integrasi OCR, Prediksi Label)

4. Simpan & Push

`git add .`

`git commit -m "feat: implementasi fungsi ekstraksi teks ocr"`

`git push origin feat/ai-aulia`

- **Penting:** Jika mengubah `models.py`, segera kabari Aziz agar tidak terjadi konflik besar. Setelah beres, buat Pull Request ke `develop`.
- Push sesering mungkin ke branch fitur masing-masing tapi pastikan tidak ada error, test dulu jalan atau tidak baru lakukan commit
- Lakukan Pull Request (PR) setelah menyelesaikan satu fungsi (misal: fungsi `upload_file`).

Tips untuk Semua (Golden Rules):

1. **Jangan Pernah Koding di Branch `develop` atau `main`.** Selalu di branch `feat/` masing-masing.
2. **Pull Sebelum Push.** Selalu lakukan `git merge origin/develop` sebelum kamu melakukan `push` agar kamu tahu jika ada kode teman yang konflik dengan kodemu.
3. **Jenkins Ready:** Karena ke depannya akan ada Jenkins, mulai sekarang biasakan **pesan commit yang jelas**. Jenkins biasanya akan membaca pesan commit ini untuk membuat laporan otomatis.

Git Laura

Dokumen 4: Panduan Peran Laura (Data Specialist)

Branch Utama: `feat/data-laura` | **Fokus:** Manajemen Dokumen & Tags.

Siklus Kerja Harian:

Bash

1. Masuk ke branch Data
`git checkout feat/data-laura`

2. Ambil update terbaru dari tim
`git fetch origin`
`git merge origin/develop`

3. Koding... (Logika simpan file, manajemen tags)

4. Simpan & Push
`git add .`
`git commit -m "feat: tambah fungsi filter dokumen berdasarkan tags"`
`git push origin feat/data-laura`

- **Penting:** Gunakan branch ini khusus untuk urusan data. Jika sudah selesai, ajukan Pull Request ke `develop` dan tag Aziz untuk di-review.
- Push sesering mungkin ke branch fitur masing-masing tapi pastikan tidak ada error, test dulu jalan atau tidak baru lakukan commit
- Lakukan Pull Request (PR) setelah menyelesaikan satu fungsi (misal: fungsi `upload_file`).

Tips untuk Semua (Golden Rules):

1. **Jangan Pernah Koding di Branch `develop` atau `main`.** Selalu di branch `feat/` masing-masing.
2. **Pull Sebelum Push.** Selalu lakukan `git merge origin/develop` sebelum kamu melakukan `push` agar kamu tahu jika ada kode teman yang konflik dengan kodemu.
3. **Jenkins Ready:** Karena ke depannya akan ada Jenkins, mulai sekarang biasakan **pesan commit yang jelas**. Jenkins biasanya akan membaca pesan commit ini untuk membuat laporan otomatis.

GIT

- a. Fitur Registrasi Akun aziz
- b. Fitur Login aziz
- c. Fitur Upload Dokumen aulia
- d. Fitur Klasifikasi Dokumen Otomatis aulia
- e. Fitur Penyimpanan Dokumen aulia
- f. Fitur Melihat Daftar Dokumen laura
- g. Fitur Pencarian Dokumen laura
- h. Fitur Edit/Update Metadata Dokumen laura
- i. Fitur Hapus Dokumen laura
- j. Fitur Dashboard Dokumen laura



Dokumentasi Pembagian Modul Proyek

Nama Modul	Branch Utama	Penanggung Jawab	Cakupan Fitur (FE & BE)
Modul 1: Infrastructure & Auth	feat/infra-aziz	Aziz	1. Registrasi Akun 2. Login & Session 3. Integrasi Object Storage & Database
Modul 2: AI Pipeline	feat/ai-aulia	Aulia	1. Upload Dokumen & Validasi 2. Trigger OCR (Ekstraksi Teks) 3. Logika Klasifikasi SVM
Modul 3: Data Management	feat/data-laura	Laura	1. Dashboard (Statistik) 2. Daftar Dokumen & Search 3. Edit Metadata & Soft-delete



Struktur Branch Git

Terdapat **5 branch** aktif selama masa pengembangan:

- main**: Branch produksi (hanya berisi kode yang sudah siap demo/sidang).
- develop**: Branch integrasi (tempat semua modul digabungkan).

3. **feat/infra-aziz**: Tempat Aziz bekerja.
4. **feat/ai-aulia**: Tempat Aulia bekerja.
5. **feat/data-laura**: Tempat Laura bekerja.

Alur Kerja Git (Workflow)

Ikuti langkah-langkah ini agar kode tetap rapi dan tidak terjadi konflik yang parah.

1. Persiapan Awal (Checkout)

Sebelum mulai koding, pastikan berada di branch masing-masing yang ditarik dari **develop**:

Bash

```
# Update kode terbaru dari server
git checkout develop
git pull origin develop
```

```
# Pindah ke branch modul sendiri
git checkout feat/nama-modul-kamu
```

2. Siklus Koding (Commit & Push)

Lakukan koding (FE & BE) di modul masing-masing. Jika sudah ada progres, simpan dan kirim ke server:

Bash

```
git add .
git commit -m "Selesaikan fitur upload dan validasi"
git push origin feat/nama-modul-kamu
```

3. Penggabungan (Pull Request & Merge)

Jangan langsung menggabungkan kode ke **develop**. Gunakan fitur **Pull Request (PR)** di GitHub/GitLab:

1. Buka repository di browser.
2. Klik **"New Pull Request"**.
3. Pilih *source*: **feat/nama-modul-kamu** → *destination*: **develop**.
4. **Aziz (sebagai Lead)** akan melakukan *Code Review*.
5. Jika sudah oke, klik **"Merge Pull Request"**.

4. Sinkronisasi (Agar Selalu Update)

Jika teman lain sudah melakukan *merge* ke **develop**, kamu harus memperbarui branch-mu agar tidak tertinggal:

Bash

```
git checkout develop
git pull origin develop
git checkout feat/nama-modul-kamu
```


git merge develop



Catatan Penting untuk Aulia & Laura

- **Mandiri FE & BE:** Kalian bertanggung jawab membuat tampilan (HTML/CSS/JS) sekaligus logika di server (Python/Node/PHP) untuk fitur tersebut.
- **Jangan Ubah Folder Orang Lain:** Sebisa mungkin koding di dalam file atau folder yang sudah disepakati untuk modul kalian agar tidak terjadi *merge conflict*.
- **Komunikasi:** Jika ingin mengubah struktur database yang bersifat global, wajib diskusi dengan Aziz.

Dokumentasi ini menggunakan asumsi struktur folder standar: `/backend` (FastAPI) dan `/frontend` (Vue + Tailwind).

Bagian 1: Finalisasi Project Setup (Sisi Aziz)

Pastikan struktur folder kamu rapi agar teman-teman tidak bingung:

Plaintext

```
my-project/
├── backend/      # Python FastAPI
│   ├── main.py
│   ├── requirements.txt
│   └── .env
├── frontend/    # Vue.js + Tailwind
│   ├── src/
│   ├── package.json
│   └── .env
└── .gitignore   # Wajib ada!
```

1. Setup Repository & SSH (GitHub/GitLab)

Jika kamu belum punya SSH key atau belum menghubungkan repo:

Generate SSH Key:

Bash

```
ssh-keygen -t ed25519 -C "email_kamu@example.com"
```

Tekan enter terus sampai selesai

-
- **Ambil kodenya:** Buka file `~/.ssh/id_ed25519.pub`, copy isinya, dan masukkan ke **Settings > SSH Keys** di akun GitHub/GitLab kamu.

Inisialisasi Repo Private:

Bash

```
git init
```

```
git remote add origin git@github.com:username/nama-repo.git
```

```
git add .
```

```
git commit -m "Initial commit: Project Structure"
```

```
git branch -M develop
```

```
git push -u origin develop
```

-
-

Bagian 2: Panduan Cloning & Setup (Untuk Aulia & Laura)

Kirimkan instruksi ini ke mereka agar mereka bisa mulai koding tanpa merusak sistem.

1. Setup Akses (Sekali Saja)

Aulia/Laura harus mengirimkan **Public SSH Key** mereka ke kamu agar kamu bisa menambahkan mereka sebagai **Collaborator** di repo private tersebut.

Note: Aulia/Laura perlu buat SSH key sendiri (seperti langkah di atas) dan berikan isi file `.pub` ke kamu.

2. Cloning Repository

Buka terminal/CMD di folder kerja mereka:

```
Bash
git clone git@github.com:username/nama-repo.git
cd nama-repo
git checkout develop
```

3. Running Backend (FastAPI)

Aulia dan Laura perlu menginstall environment Python:

```
Bash
cd backend
python -m venv venv
```

Aktifkan venv

Windows: `venv\Scripts\activate` | Mac/Linux: `source venv/bin/activate`

```
pip install -r requirements.txt
uvicorn main:app --reload
```

4. Running Frontend (Vue + Tailwind)

```
Bash
cd ../frontend
npm install
npm run dev
```



Bagian 3: Workflow Harian (Git SOP)

Agar kodingan mereka masuk ke branch yang benar (`feat/ai-aulia` atau `feat/data-laura`):

Langkah Awal (Pindah ke Branch Sendiri)

```
Bash
git checkout develop
git pull origin develop
git checkout -b feat/ai-aulia # (Aulia pakai ini)
# atau
git checkout -b feat/data-laura # (Laura pakai ini)
```

Saat Ingin Simpan Kerja (Push)

```
Bash
git add .
git commit -m "Fitur: Selesai Integrasi SVM"
git push origin feat/ai-aulia
```

Saat Ingin Mengambil Update dari Aziz

Jika kamu (`develop`) ada update fitur baru, mereka harus menariknya agar kode mereka tetap sinkron:

```
Bash
git checkout develop
git pull origin develop
git checkout feat/nama-modul-mereka
git merge develop
```



Tips untuk Aziz (Lead)

Karena mereka baru belajar koding web, pastikan kamu sudah menyiapkan file `.env.example` di folder backend dan frontend. Isinya adalah daftar variabel (seperti API Key atau DB URL) yang harus mereka isi sendiri di laptop masing-masing tanpa harus mengunggah file `.env` asli ke Git.

<https://gemini.google.com/app/26fe6b3b9bdea1e9>

<https://gemini.google.com/share/098a6d401b7e>

Untuk pengembangan yang efektif, pembagian infrastrukturnya paling ideal menggunakan pola **"Local First, Staging Central"**.

Artinya, setiap orang punya "dapur" sendiri di laptop masing-masing, tapi ada satu "ruang pameran" bersama di server. Berikut rinciannya:

1. Di Laptop Masing-Masing (Local Development)

Setiap orang (Aziz, Aulia, Laura) menjalankan **seluruh stack** secara lokal menggunakan **docker-compose**.

- **FE & BE:** Berjalan di laptop sendiri agar bisa koding dan *debugging* dengan cepat tanpa gangguan koneksi internet.
- **PostgreSQL & MinIO:** Setiap orang punya kontainer database dan storage sendiri.
 - *Keuntungan:* Jika Aulia menghapus data saat tes AI, database di laptop Laura tidak akan ikut terhapus. Mereka bebas melakukan eksperimen apa pun.

2. Di Server Ubuntu (Staging Central - Tailscale)

Gunakan IP **10.0.2.129** sebagai **Server Integrasi**.

- **Database & MinIO Pusat:** Di sini tersimpan data "asli" atau data hasil pengujian bersama.
- **Deployment:** Begitu fitur dari Aulia atau Laura sudah kamu *merge* ke branch **develop**, kamu *deploy* ke server ini agar semua orang bisa melihat hasil akhirnya di satu URL yang sama.

Cara Mengelola PostgreSQL & MinIO agar Sinkron:

Komponen	Cara Kerja di Laptop (Local)	Cara Kerja di Server (Staging)
PostgreSQL	Menggunakan Docker (data disimpan di volume lokal).	Database pusat yang menampung data dari semua modul.
MinIO	Menggunakan Docker (folder lokal) untuk simpan file dummy saat tes.	Menggunakan NFS Mount (/mnt/staging_sad22) untuk penyimpanan file asli.
Config (.env)	Mengarah ke localhost:5432 dan localhost:9000 .	Mengarah ke IP Tailscale 10.0.2.129 dan port staging.

Rekomendasi Alur untuk Tim:

1. **Aziz** siapkan file `docker-compose.yml` yang sudah berisi konfigurasi Postgres dan MinIO.
2. **Aulia & Laura** cukup jalankan `docker-compose up -d`. Mereka langsung punya server BE, FE, DB, dan S3 sendiri di laptopnya.
3. **Sinkronisasi**: Jika ada perubahan struktur tabel database (migration), kamu yang memimpin untuk update file SQL-nya agar mereka tinggal *pull* dan jalankan.

Pesan untuk Tim: "Kalian koding dan tes di laptop masing-masing sampai jalan. Kalau sudah oke, lapor Aziz, nanti Aziz bantu naikkan ke server staging (Tailscale) buat kita liat bareng-bareng."

1. Update Repository & Install

Buka terminal WSL Ubuntu kamu dan jalankan perintah berikut:

Bash

Update daftar paket

`sudo apt update`

Install Git

`sudo apt install git -y`

2. Verifikasi Instalasi

Pastikan Git sudah terinstall dengan mengecek versinya:

Bash

`git --version`

3. Konfigurasi Identitas (Wajib)

Karena kamu akan melakukan *commit* ke branch `feat/infra-aziz`, kamu harus mengatur nama dan email agar terbaca di GitHub:

Bash

```
git config --global user.name "Umar Abdul Aziz"
```

```
git config --global user.email "email_kamu@example.com"
```

4. Tips: Integrasi Git Credential (Opsional tapi Penting)

Agar kamu tidak perlu mengetik *username/password* atau *token* setiap kali **push** dari WSL, kamu bisa menyambungkannya dengan **Git Credential Manager** milik Windows:

Bash

```
git config --global credential.helper "/mnt/c/Program\
Files/Git/mingw64/bin/git-credential-manager.exe"
```

(Catatan: Ini berasumsi kamu sudah menginstall Git for Windows di sisi Windows-nya).

Pro-Tip untuk Aziz:

Mengingat kamu menggunakan **SSH** untuk repo privat (seperti yang kita bahas sebelumnya), jangan lupa untuk membuat SSH Key **di dalam WSL** jika belum ada:

1. Jalankan `ssh-keygen -t ed25519 -C "email@kamu.com"`.
2. Ambil kodenya dengan `cat ~/.ssh/id_ed25519.pub`.
3. Masukkan ke GitHub Settings.

Readme.md

Sistem Klasifikasi & Pengarsipan Dokumen Digital (SAD22)

Proyek Sistem Analisis dan Desain (SAD) untuk otomatisasi pengarsipan dokumen digital menggunakan Machine Learning (SVM) dan penyimpanan berbasis S3 (MinIO).

Tech Stack & Pembagian Modul

1. Modul Infrastructure & Auth (PIC: Umar Abdul Aziz)

- **Backend:** FastAPI (Python), JWT Auth, S3 Integration.
- **DevOps:** Docker, Docker Compose, Tailscale Networking.
- **Database:** PostgreSQL (Metadata Storage).

2. Modul AI Pipeline (PIC: Aulia Fatiha Sari)

- **AI/ML:** Scikit-Learn (SVM), Joblib.
- **OCR:** Tesseract/EasyOCR (Ekstraksi Teks).
- **Frontend:** Vue 3 + TypeScript (Halaman Upload & Validasi).

3. Modul Data Management (PIC: Ismi Laura)

- **Frontend:** Vue 3 + TypeScript (Dashboard & Search UI).
- **State Management:** Pinia.
- **Storage Logic:** Soft-delete mechanism & Search Indexing.

Struktur Proyek

Plaintext

```
arsip-sad-app/
├── frontend/           # Vue.js + TypeScript Application
│   ├── src/           # Source code utama
│   ├── Dockerfile      # Script build Vue ke Nginx
│   ├── package.json    # Dependensi & Scripts
│   └── vite.config.ts  # Konfigurasi Vite
├── backend/           # FastAPI Application
│   ├── app/           # Router, DB Connection, & S3 Logic
│   ├── main.py         # Entry point API
│   ├── requirements.txt # Library Python
│   └── Dockerfile      # Script build Python/FastAPI
└── ml_model/          # Machine Learning Research & Assets
```

		model_svm.joblib	# Binary model hasil training
		train.py	# Script training/retraining
		docker-compose.yml	# Orchestration (DB, Storage, App)
		.gitignore	# Ignored files (node_modules, .env, dll)
		README.md	# Dokumentasi Proyek

Panduan Pengembangan

1. Persiapan Environment

Salin file `.env.example` menjadi `.env` di folder `backend/` dan `frontend/`, lalu sesuaikan konfigurasinya.

2. Menjalankan via Docker (Rekomendasi)

Bash

```
docker-compose up -d --build
```

3. Menjalankan Manual (Lokal)

- **Frontend:** `cd frontend && npm install && npm run dev`
 - **Backend:** `cd backend && pip install -r requirements.txt && uvicorn main:app --reload`
-

Aturan Branching & Kolaborasi

Kami menggunakan strategi **Git Flow (Simplified)**. Dilarang melakukan `push` langsung ke `main` atau `develop`.

Daftar Branch Utama:

- `main` : Kode stabil untuk produksi/sidang.
- `develop` : Branch utama untuk penggabungan fitur (Integrasi).
- `feat/infra-aziz` : Pengembangan Modul 1.
- `feat/ai-aulia` : Pengembangan Modul 2.
- `feat/data-laura` : Pengembangan Modul 3.

Alur Kerja (Workflow):

1. **Sync:** `git checkout develop -> git pull origin develop`.
2. **Work:** Pindah ke branch modul masing-masing (`git checkout feat/...`).

3. **Commit:** `git add . -> git commit -m "feat: deskripsi fitur"`.
 4. **Push:** `git push origin feat/....`
 5. **Merge:** Buat **Pull Request (PR)** ke branch `develop` untuk di-review oleh Aziz.
-

Koneksi Staging (Tailscale)

- **API Staging:** `http://10.0.2.129:2000`
 - **MinIO Console:** `http://10.0.2.129:2001`
 - **NFS Mount Path:** `/mnt/staging_sad22`
-

Tim Pengembang

- **Umar Abdul Aziz** (@UmarAziz01) - *Lead Developer*
 - **Aulia Fatiha Sari** - *AI Engineer*
 - **Ismi Laura** - *System Analyst & Data Manager*
-

Apa saja yang saya ubah?

1. **Pembagian Tech Stack:** Sekarang dikelompokkan per **Modul**, bukan sekadar Frontend/Backend. Ini memperjelas siapa mengerjakan apa (FE & BE) di bagian fiturnya masing-masing.
2. **Struktur Folder:** Saya masukkan folder `ml_model/` dan `docker-compose.yml` sesuai struktur detail yang baru saja kamu berikan.
3. **Branching Strategy:** Menghapus branch `dev-nama` dan menggantinya dengan **3 Branch Modul + Branch Develop** sesuai rencana terbaru kita.
4. **Workflow Git:** Saya tambahkan langkah **Pull Request (PR)** ke `develop`, agar kamu punya kesempatan mengecek kodingan Aulia dan Laura sebelum resmi digabung.
5. **Instruksi Docker:** Karena kamu punya `docker-compose.yml`, saya tambahkan perintah `docker-compose up` sebagai cara tercepat bagi temanmu untuk menjalankan project.

ALUR SISTEM

◆ 1. Alur Sistem (End-to-End, dari awal sampai akhir)

Berikut alur sistem kamu (Arsip Pintar) yang sudah dirapikan dari SRS:

● 1. Autentikasi

◆ Register

1. User input:
 - email
 - password
2. Backend:
 - Validasi format email
 - Hash password (Argon2id)
 - Simpan ke tabel **users**
3. Akun berhasil dibuat dan Login

◆ Login

1. User input email & password
2. Backend:
 - Ambil data user
 - Verifikasi hash
 - Generate JWT / session
3. User masuk ke dashboard

● 2. Upload Dokumen

1. User klik upload atau bisa drag and drop (bisa beberapa file secara bersamaan)
2. Sistem:
 - Validasi format (PDF & DOC)
 - Simpan file di MinIO
 - Simpan metadata pada Postgree

● 3. Klasifikasi (Core Sistem)

Fitur ini berjalan dibelakang layar, setelah user upload dokumen maka otomatis sistem mengekstraksi dan klasifikasi lalu user bisa mengkonfirmasi hasil klasifikasi dari AI. Atau jika tidak sesuai bisa mengubah dengan label yang diinginkan.

1. OCR membaca isi teks dokumen menggunakan PyMuPDF, atau dengan opsi kedua yaitu tesseract ocr jika PyMuPDF tidak bisa melakukan OCR. Lalu disimpan di Postgree dengan format TSVector.
 2. TF-IDF mengubah teks → numerik
 3. SVM melakukan klasifikasi label dengan model.pkl
 4. Sistem menghasilkan:
 - Label dokumen (KRS, KHS, Sertifikat, Transkrip, dll)
 - Metadata dokumen
 - Auto rename file dengan format [yyyy_mm_dd_label_id_namafile] Variabel tahun, bulan, dan tanggal diambil dari tanggal upload dokumen.
-

4. Penyimpanan Dokumen

File yang muncul hanya jika user telah mengkonfirmasi hasil klasifikasi (Berstatus “Confirmed”). File dengan hasil akurasi >60 akan berstatus “Classified”. Apabila Hasil akurasi <60 maka dokumen akan berstatus “Needs Review” dimana dokumen akan dikembalikan kepada user untuk memberi label yang sesuai.

1. Sistem menyimpan:
 - File ke object storage (MinIO)
 - Metadata ke database (PostgreSQL)
 - User dapat mengunduh dokumen yang sudah tersimpan ketika diperlukan
 2. File dienkripsi (AES-256)
-

5. Manajemen Dokumen (CRUD)

User bisa:

- Lihat daftar dokumen (ada tab sesuai label)
 - Edit metadata (nama file sistem, label, tag)
 - Hapus (soft delete → masuk trash) Jika file melampaui 30 hari pada sampah, maka otomatis akan terhapus secara permanen pada frontend dan backend.
-

6. Pencarian Dokumen

1. User input keyword
2. Sistem:

- Full-text search (GIN + stemming)
 - Cari di:
 - Metadata (Label, Nama file sistem, Tag)
 - Isi teks dari hasil OCR
 - 3. Tampilkan hasil
-

7. Dashboard

1. Sistem mengambil data statistik (Jumlah persebaran data untuk setiap label dan sisa kuota penyimpanan). Setiap user memiliki kuota penyimpanan sebesar 10gb.
 2. Menampilkan:
 - Data statistik persebaran jumlah dokumen sesuai label
 - Statistik kuota penyimpanan
-

DIAGRAM

Pembagian Tugas

DL MENGERJAKAN SEMUA DOKUMEN INI KAMIS PAGI JAM 9, LALU SETELAH ITU DIREVIEW BERSAMA SIANG HARI

SETIAP ANGGOTA HARUS MENGUPLOAD SEMUA DIAGRAM DI DRAW IO [link](https://app.diagrams.net/#G1dc8MpgRVtE664CPTQIAFVhH8AxOQEzjz)
<https://app.diagrams.net/#G1dc8MpgRVtE664CPTQIAFVhH8AxOQEzjz>

Anggota 1: Ismi Laura Azakkiyyah (Fokus: Autentikasi & Visualisasi) Menangani fitur yang berinteraksi langsung dengan identitas pengguna dan ringkasan data.

- Fitur Registrasi Akun
- Fitur Login
- Fitur Dashboard Dokumen
- Data Flow Diagram
- Deployment Diagram

Anggota 2: Aulia Fatiha Sari (Fokus: Pemrosesan Inti & Integrasi AI) Menangani alur otomatisasi dan pemrosesan yang berjalan di belakang layar (*background process*). Karena fitur ini melibatkan logika yang kompleks (seperti OCR dan ML) dan penyimpanan persisten, ketelitian dalam merancang alur teknisnya sangat krusial.

- Fitur Upload Dokumen (termasuk validasi dan *trigger* OCR)
- Fitur Klasifikasi Dokumen Otomatis (logika AI)
- Fitur Penyimpanan Dokumen (ke *object storage* dan *database*)
- Class Diagram

Anggota 3: Umar Abdul Aziz (Fokus: Manajemen Dokumen & Operasional) Menangani antarmuka operasional (CRUD) lanjutan setelah dokumen berhasil disimpan.

- Fitur Melihat Daftar Dokumen
- Fitur Pencarian Dokumen
- Fitur Edit/Update Metadata Dokumen
- Fitur Hapus Dokumen (*soft-delete*)
- Software Architecture

Software Architecture Kurang Aut, penyimpanan dokumen,

HASIL WAWANCARA

KAK TYA

Informasi Wawancara

- **Nama Narasumber:** Awanda Fitya Zahra
 - **Semester Saat Ini atau Status:** 8
 - **Tanggal Wawancara:** Senin, 16 Maret 2026
 - **Pemandu Wawancara:** Umar Abdul Aziz
 - **Observer & Notulis:** Ismi Laura Azakiyyah dan Aulia Fatiha Sari
-

Hasil Wawancara Berdasarkan Daftar Pertanyaan

1. Pengelolaan Dokumen Saat Ini

- **Cara Menyimpan Dokumen:** Mbak Tya menggunakan beberapa platform terpisah, seperti Google Drive, email, hingga membiarkan *file* masih tersimpan di pihak panitia penyelenggara. Ia menggunakan akun email kampus (UINSA) karena memiliki kapasitas penyimpanan (*storage*) yang sedikit lebih besar daripada Gmail biasa, mengingat Google Drive gratis hanya berkapasitas 15 GB.
- **Cara Mengatur/Mengelompokkan:** Pengelolaannya masih manual dengan cara menjadikan *file* ke dalam satu *folder* spesifik. Namun, keterbatasan *storage* membuat dokumen akademik sering tercampur dengan dokumen pribadi atau dokumentasi lainnya.

2. Permasalahan yang Pernah Dialami

- **Kesulitan Mencari Dokumen Dadakan:** Ia sangat kesulitan saat mendadak harus mengisi SKEK karena sertifikat dari semester 1 (lebih dari 3 tahun lalu) sangat sulit dicari. Ia juga pernah kesulitan mencari dua sertifikat Ma'had; karena hanya dikirim satu lewat email, ia akhirnya menghubungi pihak PUSTIPD untuk mengirimkan sisanya lewat WhatsApp. Saat sertifikat *bootcamp* UGM hilang, ia harus menghubungi *contact person* panitia di pamflet acara.
- **Kendala Paling Sering:** Ruang penyimpanan yang terbatas sering memaksanya menghapus *file* di penyimpanan lokal (HP/Laptop) agar memori tidak penuh. Selain itu, sertifikat lama yang dikirim via email rawan hilang karena terhapus atau karena ia lupa mengecek email.

3. Pengalaman Administrasi Akademik

- **Keperluan Mengumpulkan Dokumen:** Di semester akhir, dokumen sangat banyak diminta untuk pendaftaran Sempro, Semhas, Laporan Magang, Sidang Akhir, serta Yudisium.
- **Dokumen yang Sering Diminta:**
 - **Sempro:** Mewajibkan *file* Bab 1 hingga Bab 3 yang diunggah terpisah per bab.
 - **Laporan Magang:** Memerlukan *logbook*, laporan magang, ceklis tanda tangan, dan absensi.
 - **Semhas & Sidang Akhir:** Membutuhkan transkrip nilai, SKEK yang ditandatangani Wakil Dekan, KTP, Ijazah (mulai dari SMA), serta bukti bayar UKT.
 - **Yudisium:** Mensyaratkan surat bebas perpustakaan (bukti unggah ke Digilib) dan surat bebas lab. Ada juga syarat tambahan berupa surat penghapusan email kampus UINSA.

4. Jenis Dokumen yang Sering Dibutuhkan

- Sebagai pendaftar beasiswa, dokumen yang paling sering dan berulang kali diminta darinya adalah **KHS dan Surat Aktif Kuliah**. Kedua dokumen ini memiliki masa berlaku

(*kadaluarsa*) hanya selama 3 bulan setelah penerbitan. Selain itu, **sertifikat kegiatan** sangat dibutuhkan dalam jumlah besar karena mahasiswa diwajibkan mengumpulkan minimal 60 poin SKEK, sedangkan satu sertifikat anggota kepanitiaan hanya bernilai 2 poin.

5. Validasi Solusi Sistem

- **Perluakah Sistem Manajemen Digital?:** Sangat perlu, agar dokumen akademik lebih terorganisir dan tidak tercampur dengan dokumen pribadi. sehingga mahasiswa bisa langsung merujuk ke satu aplikasi khusus.
- **Kegunaan Fitur Pengelompokan Otomatis:** Fitur penyortiran sangat krusial, khususnya untuk memilah sertifikat berdasarkan kategori SKEK (Aspek dan Sub-aspek, seperti Loyalitas atau Idealisme). Pengelompokan ini penting untuk menghindari kesalahan validasi. Mbak Tya pernah memiliki **23 poin SKEK yang ditolak oleh Dosen Wali** (yang sangat teliti) karena ia memasukkan sertifikat webinar internasional senilai 7 poin yang topiknya tentang pendidikan anak (tidak masuk *scope* keilmuan Sistem Informasi), serta karena ia mencoba memasukkan poin dari artikel terbitan milik teman-temannya.

6. Kebutuhan Fitur dan Keamanan

- **Fitur Paling Penting:** Mbak Tya menyarankan agar aplikasi dibuat berbasis *website* atau *extension* peramban alih-alih aplikasi *mobile* agar tidak banyak memakan kapasitas penyimpanan gawainya. Ia juga sangat menyarankan penambahan fitur *auto-upload* (unggah otomatis) pada *extension* tersebut ketika mahasiswa mengunduh sertifikat, sehingga dokumen langsung tersimpan ke sistem.
- **Keamanan Data:** Meskipun ia memiliki tingkat kepercayaan sekitar 80% terhadap layanan pihak ketiga seperti Google Drive,, ia sangat khawatir jika data pribadinya bocor, seperti kasus kebocoran data BPJS beberapa tahun lalu. Ia menegaskan bahwa **fitur enkripsi sangat wajib digunakan** agar data tetap aman dan pihak *developer* aplikasi sekalipun tidak bisa mengakses isi dokumen mahasiswa.

Saran Stakeholder (Insight Tambahan di Luar Pertanyaan Wawancara)

1. **Antisipasi Migrasi Data Akibat Penghapusan Email Kampus:** Terdapat *insight* baru bahwa email kampus UINSA yang sering dimanfaatkan penyimpanannya oleh mahasiswa akan dihapus setelah mereka lulus (harus ada proses pembuatan surat penghapusan email kampus). Aplikasi ini disarankan memiliki **fitur migrasi atau backup data (export file)** secara masif ke *drive* pribadi, sehingga *file* mahasiswa yang sebelumnya tertaut di email kampus tidak hangus setelah lulus.
2. **Integrasi Single-Sign-On (SSO) dan Auto-Upload dengan "Sinau":** Sistem sangat disarankan agar terhubung dengan sistem Sinau. Sistem bisa menggunakan data login secara otomatis (*auto-isi*). Selain itu, aplikasi ini diharapkan dapat menjadi **sistem cadangan saat server Sinau sedang mengalami down**.
3. **Fleksibilitas Penggandaan Dokumen (Bercermin dari Sistem "Amira"):** Sistem diharapkan bisa meniru fleksibilitas sistem Amira. Di Amira, mahasiswa dapat mengunduh *soft file* dokumen, lalu mencetaknya berkali-kali secara mandiri untuk kemudian dimintakan tanda tangan basah secara fisik sesuai kebutuhan.

KAK SAFRIYA

Informasi Wawancara

- **Nama Narasumber:** Safriya Murni Puspita
 - **Semester Saat Ini atau Status:** Lulus
 - **Tanggal Wawancara:** Selasa, 17 Maret 2026
 - **Pemandu Wawancara:** Aulia Fatiha Sari
 - **Observer & Notulis:** Umar Abdul Aziz dan dan Ismi Laura Azakiyyah
-

Hasil Wawancara Berdasarkan Daftar Panduan

1. Pengelolaan Dokumen Saat Ini

- **Cara Menyimpan Dokumen:** Selama perkuliahan, Mbak Safriya biasanya mengumpulkan sertifikat-sertifikat ke dalam satu folder di **Google Drive dengan menggunakan akun email kampus**.
- **Cara Mengatur/Mengelompokkan:** Pihak kampus atau jurusan biasanya hanya meminta tautan (*link*) Google Drive dari akun kampus tersebut sebagai bukti pengumpulan, sehingga ia tidak mengunggah (*upload*) *file* dokumennya secara langsung satu per satu ke sistem kampus.

2. Permasalahan yang Pernah Dialami

- **Kesulitan Mencari Dokumen:** Ia menyoroti bahwa sertifikat sangat rawan hilang karena panitia atau mentor sering mengirimkannya melalui berbagai *platform* yang berbeda. Ada yang mengirim via Google Drive, via email, hingga membagikan *link* melalui Grup WhatsApp.
- **Kendala Paling Sering:** Grup WhatsApp memiliki kelemahan di mana pesan atau tautan yang sudah lama akan tertumpuk, terhapus, atau hilang saat dicari kembali. Pengiriman via email pun kadang rawan terlewat atau hilang. Akibatnya, mahasiswa kebingungan melacak di mana sertifikat mereka dulu dikirimkan.

3. Pengalaman Administrasi Akademik

- **Keperluan Mengumpulkan Dokumen:** Dokumen akademik sangat banyak diminta terutama menjelang akhir perkuliahan, seperti untuk syarat pengajuan Sempro, Semhas, dan Skripsi.
- **Dokumen yang Sering Diminta:** Untuk pendaftaran skripsi, persyaratan yang diminta antara lain Kartu Keluarga (KK), KTP, surat pengesahan, surat bebas perpustakaan, dan surat bebas laboratorium. Namun, ia mencatat bahwa surat bebas perpustakaan dan lab biasanya langsung diurus dan dikeluarkan pada saat itu juga, sehingga tidak perlu disimpan secara intens sejak awal.

4. Jenis Dokumen yang Sering Dibutuhkan (dan Paling Rawan Hilang)

- Dokumen yang sangat krusial, sering dibutuhkan untuk syarat kelulusan/skripsi, dan paling sering hilang adalah **sertifikat wajib kampus**. Ini meliputi: **Sertifikat IBA**, **Sertifikat IBI (seperti TOEFL/TOAFL)**, **Sertifikat Baca Tulis Al-Qur'an (BTQ)**, **Sertifikat Ma'had**, dan **Sertifikat ICT**.

5. Validasi Solusi Sistem

- **Perluah Sistem Manajemen Digital?:** Sangat perlu dan ide ini dinilai bagus. Mbak Safriya sangat menyarankan agar sistem yang dibangun difokuskan (dikerucutkan) untuk menyelamatkan dokumen-dokumen krusial (seperti sertifikat IBA-IBI, ICT, BTQ) yang memang rawan hilang tersebut.
- **Kegunaan Fitur Pengelompokan:** Dibandingkan sistem unggah (*upload*) manual untuk dikelompokkan, ia lebih menyarankan agar sistem diotomatisasi. Jika fitur berupa label "Sertifikat Wajib", mahasiswa tidak akan khawatir lagi sertifikatnya tercecer di WhatsApp.

6. Kebutuhan Fitur dan Keamanan

- **Fitur Paling Penting (Otomatisasi Email):** Mbak Safriya memberikan masukan krusial agar sistem pengarsipan ini **langsung dihubungkan (terintegrasi) dengan email kampus mahasiswa**. Jadi, ketika admin atau panitia mengirimkan sertifikat ke email kampus, sistem akan secara otomatis menarik data tersebut, mengidentifikasinya, dan memasukkannya ke dalam folder klasifikasi di aplikasi tanpa mengharuskan mahasiswa melakukan *upload* manual.
- **Keamanan dan Privasi:** Fitur keamanan sangat penting karena beberapa dokumen memiliki skor ujian (seperti sertifikat IBA-IBI/TOEFL) yang sifatnya rahasia. Pengiriman otomatis dari admin langsung ke sistem pemiliknya akan lebih aman karena bersifat pribadi dan terpusat.

Saran Stakeholder (Insight Tambahan di Luar Pertanyaan Wawancara)

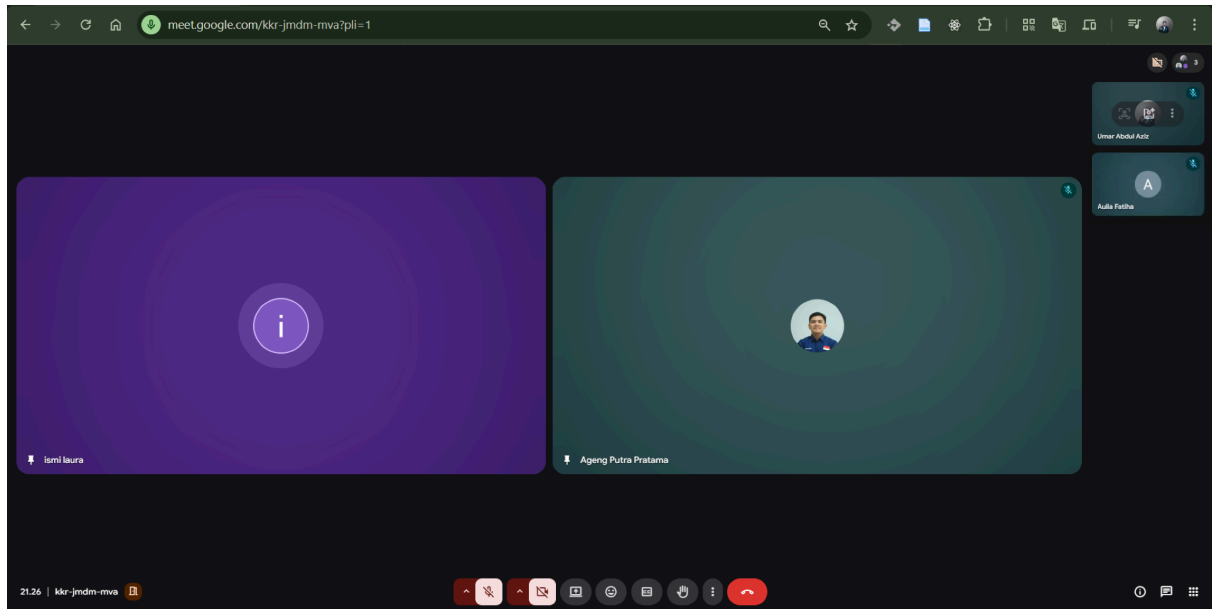
1. Kesempurnaan Akurasi Bukan Penilaian Utama Mbak Safriya membagikan pengalaman proyek SAD-nya (mendeteksi mata secara otomatis) yang secara hasil akhir sebenarnya tidak valid/tidak akurat. Namun, proyeknya tetap dianggap berhasil oleh Pak Subhan karena yang terpenting dalam mata kuliah SAD adalah **kemampuan mahasiswa dalam menganalisis alur kelemahan sistem dan bisa menjelaskan secara ilmiah mengapa sistem tersebut tidak akurat** (misalnya karena kurangnya *dataset*), bukan sekadar menciptakan aplikasi yang sempurna.

2. Evaluasi Fitur Agar Tidak Melebar Anggota kelompok (Umar dan Laura) sempat berdiskusi usai wawancara bahwa ide untuk menghubungkan sistem langsung dengan server/API email kampus mungkin cukup rumit secara teknis dan bisa membuat fokus sistem yang awalnya tentang "ekstraksi dokumen/Machine Learning" menjadi "melebar" ke ranah jaringan/API. Tim harus bijak menentukan apakah saran otomatisasi email ini bisa diterapkan (bisa dinegosiasikan dengan Pak Subhan) atau tetap mempertahankan alur awal (mengunggah manual) sambil menyiapkan argumentasi yang kuat.

KAK PRATAMA

Informasi Wawancara

- **Nama Narasumber:** Ageng Putra Pratama
- **Semester Saat Ini atau Status:** Lulus
- **Tanggal Wawancara:** Selasa, 17 Maret 2026
- **Pemandu Wawancara:** Ismi Laura Azakiyyah
- **Observer & Notulis:** Umar Abdul Aziz dan dan Aulia Fatiha Sari



Hasil Wawancara Berdasarkan Daftar Panduan

1. Pengelolaan Dokumen Saat Ini

- **Cara Menyimpan Dokumen (Sebagai *Life Tracker*):** Mas Pratama menerapkan metode penyimpanan berlapis (*backup*) di Google Drive, *flash disk*, penyimpanan lokal laptop, dan *spreadsheet*. Menariknya, ia menggunakan *spreadsheet* bukan sekadar untuk kuliah, melainkan sebagai **pencatat riwayat hidup (*life tracker*) dari kecil hingga sekarang**, lengkap dengan tautan (*link*) bukti dokumen pendukungnya.
- **Cara Mengatur/Mengelompokkan:** Ia sangat terstruktur dan secara rutin menjadwalkan ***decluttering*** (membersihkan dan memilah *file*) setiap akhir bulan atau akhir pekan. Ia mengadaptasi metode dari buku *Building a Second Brain* (Tiago Forte), yaitu konsep pengelompokan **folder PARA** (*Project, Area, Resource, Archive*) dan metode pemrosesan informasi **CODE** (*Capture, Organize, Distill, Express*). Aplikasi yang paling sering ia andalkan adalah Google Keep karena kesederhanaannya dan integrasinya dengan ekosistem Google Workspace.

2. Permasalahan yang Pernah Dialami

- **Kesulitan Mencari Dokumen:** Karena disiplin mencadangkan data di berbagai tempat, ia mengaku hampir tidak pernah mengalami kehilangan *file*.
- **Kendala Paling Sering:** Kendala yang ia alami hanyalah **penamaan *file* yang tidak konsisten** antar-*folder*, yang terkadang membuatnya sedikit kebingungan. Saat ini ia menyiasatinya dengan format penamaan standar: tahun, nama pemilik, lalu nama *file*.

3. Pengalaman Administrasi Akademik

- **Keperluan Mengumpulkan Dokumen:** Dokumen akademik biasanya dikumpulkan untuk persyaratan di semester akhir, seperti pendaftaran **Semhas, Skripsi, dan Yudisium**.
- **Dokumen yang Sering Diminta:** Dokumen meliputi KRS, transkrip nilai, lembar pengesahan, bukti bayar UKT, pas foto, serta surat bebas perpustakaan dan laboratorium. Mas Pratama mengatasinya dengan membuat folder khusus persyaratan jauh-jauh hari, sehingga saat hari-H ia hanya tinggal mengunggahnya.

4. Jenis Dokumen yang Sering Dibutuhkan

- Dokumen yang paling sering diunduh dan digunakan berulang kali adalah **Transkrip Nilai**. Dokumen ini sangat esensial terutama ketika ia ingin *apply* (melamar) program magang atau pekerjaan.

5. Validasi Solusi Sistem

- **Perlukah Sistem Manajemen Digital?:** Sangat perlu. Menurutnya, kelemahan sistem akademik kampus saat ini (Sinau) adalah tidak bisa merekam *track record* atau portofolio utuh mahasiswa. Sistem Sinau hanya mendata poin SKEK, tetapi tidak bisa memfilter pencapaian apa saja yang relevan untuk ditunjukkan saat melamar pekerjaan.

6. Kebutuhan Fitur dan Keamanan (Termasuk Inovasi)

- **Fitur Filter dan Rekomendasi AI pada CV:** Ia menyarankan sistem AI yang bisa merekomendasikan pembuatan CV secara otomatis. Jika mahasiswa melamar sebagai *Machine Learning Engineer*, sistem tidak hanya merekomendasikan sertifikat yang relevan, tetapi juga **menarik riwayat nilai course (seperti nilai *Machine Learning*) serta riwayat sistem/proyek yang berhasil dikembangkan** ke dalam CV. Sistem juga akan otomatis menyisihkan dokumen yang tidak relevan (seperti sertifikat *web development*).
- **Inspirasi Pemetaan Data Ala Obsidian:** Terinspirasi dari aplikasi Obsidian, ia menyarankan fitur *niche* berupa **visualisasi alur keterkaitan dokumen yang berbentuk seperti "saraf-saraf"**. Fitur ini memungkinkan pengguna melihat alur hubungan dari satu titik data (dokumen) ke titik data lainnya.

RINGKASAN HASIL STAKEHOLDER

1. Pengelolaan Dokumen Saat Ini

- **Tempat Penyimpanan:** Ketiga narasumber mengandalkan ekosistem Google (Drive, Keep, Email). Mbak Tya dan Mbak Safriya secara khusus memanfaatkan akun email kampus karena kapasitasnya yang lebih besar.
- **Metode Pengaturan:** Mas Pratama sangat terstruktur dengan sistem *backup* berlapis dan metode decluttering PARA/CODE. Sebaliknya, Mbak Tya dan Mbak Safriya masih mengandalkan pembuatan folder manual, yang seringkali membuat *file* akademik tercampur dengan *file* pribadi akibat keterbatasan ruang.

2. Permasalahan yang Pernah Dialami

- **Kehilangan & Pencarian:** Mahasiswa sangat kesulitan mencari dokumen lama (seperti sertifikat semester 1) saat mendadak butuh untuk SKEK. Mbak Safriya menyoroti bahwa sertifikat sangat rawan hilang karena panitia mengirimnya via berbagai platform (WhatsApp, Email, Drive), di mana pesan WhatsApp sering tertumpuk/terhapus. Mbak Tya bahkan sampai harus menghubungi Admin Pusat Universitas (PUSTIPD) dan panitia acara untuk meminta ulang dokumen yang hilang.
- **Inkonsistensi:** Mas Pratama jarang kehilangan *file*, namun sering kebingungan akibat penamaan *file* yang tidak konsisten antar-folder.

3. Pengalaman Administrasi Akademik

- **Fase Kritis:** Kebutuhan dokumen memuncak di semester akhir untuk keperluan Sempro, Semhas, Laporan Magang, Sidang Akhir, dan Yudisium.
- **Dokumen yang Wajib Ada:** Transkrip nilai, KRS, KTP, Ijazah, bukti bayar UKT, hingga lembar pengesahan. Terdapat juga syarat administratif wajib seperti surat bebas perpustakaan dan laboratorium.

4. Jenis Dokumen yang Sering Dibutuhkan

- **Sertifikat Wajib & SKEK:** Sertifikat (IBA, IBI/TOEFL, BTQ, Ma'had, ICT, kepanitiaan) adalah dokumen paling krusial, sering dibutuhkan untuk syarat kelulusan, dan paling rawan hilang.
- **Dokumen Berulang & Kedaluwarsa:** Transkrip Nilai sering diunduh berulang kali untuk melamar magang/kerja. Sementara KHS dan Surat Aktif Kuliah sering dibutuhkan untuk beasiswa namun memiliki masa berlaku singkat (3 bulan).

5. Validasi Solusi Sistem

- **Tingkat Kebutuhan:** Ketiganya sepakat bahwa sistem ini sangat perlu, mengingat proses administrasi akhir studi (seperti pendaftaran yudisium dan wisuda) sangat bergantung pada ketersediaan dokumen akademik yang saat ini sering kali tidak terorganisasi dan masih tercampur dengan berkas pribadi.
- **Fungsi Klasifikasi:** Sistem memerlukan fungsionalitas penyortiran dokumen yang terstruktur. Terdapat urgensi untuk mengklasifikasikan sertifikat berdasarkan kategori SKEK guna meminimalisasi risiko penolakan poin validasi oleh Dosen Wali, sebagaimana insiden penolakan 23 poin yang pernah terjadi

6. Kebutuhan Fitur dan Keamanan

- Fitur Penting yang mungkin perlu di kembangkan:
- **Keamanan Ekstra:** Mengingat dokumen bersifat privat (seperti skor TOEFL), diperlukan enkripsi agar developer sekalipun tidak bisa mengakses data pengguna, guna mencegah kebocoran data.

7. Saran Stakeholder (Insight Tambahan)

- Aplikasi ini difungsikan sebagai sistem cadangan (backup) sekunder bagi mahasiswa untuk mengakses dokumen akademik seperti KRS, KHS, dan sertifikat. Dalam kondisi ketika peladen (server) sistem akademik kampus (Sinau) sedang mengalami gangguan atau downtime, mahasiswa tetap dapat mengunduh dokumen-dokumen krusial tersebut melalui aplikasi ini, dengan prasyarat bahwa berkas terkait telah diunggah ke dalam sistem sebelumnya.

RINGKASAN HASIL USER

Wawasan (*Insight*) Berdasarkan Setiap Pertanyaan Kuesioner

A. Analisis Masalah dan Kebiasaan Saat Ini (Kualitatif & Skala)

- **Q1: Frekuensi Kesulitan Menemukan Dokumen (Skala 1-5):** Mayoritas responden memilih angka 3 (terkadang) hingga 4 (sering) dan 5 (selalu).
 - **Insight Validasi:** Masalah kehilangan atau kesulitan melacak arsip akademik adalah masalah nyata dan persisten di kalangan mahasiswa. Kebutuhan akan aplikasi arsip ini tidak dibuat-buat, melainkan berdasarkan *pain point* (titik masalah) yang sangat valid di lapangan.
- **Q2: Tindakan saat Kesulitan Menemukan Dokumen:** Responden menjawab mencari satu per satu di semua *folder*, *recycle bin*, bertanya pada teman, hingga menghubungi pihak akademik/PUSTIPD.
 - **Insight Validasi:** Mahasiswa masih sangat bergantung pada pihak ketiga dan proses pencarian manual yang memakan waktu. Ini memvalidasi urgensi fitur pencarian pintar berbasis kata kunci untuk menciptakan kemandirian pengguna.
- **Q3: Cara Menyimpan Dokumen Saat Ini:** Jawaban bervariasi mulai dari Google Drive, laptop, diunduh di memori HP, dibiarkan tidak teratur, hingga ada yang dicetak dan disimpan dalam lemari/rak/map.
 - **Insight Validasi:** Penyimpanan mahasiswa saat ini sangat terdesentralisasi (berceceran) dan tidak memiliki standar. Hal ini mendukung temuan wawancara bahwa dokumen akademik sering tercampur dengan dokumen pribadi, sehingga sistem yang terpusat dan terdedikasi sangat dibutuhkan.

B. Analisis Kebutuhan Kualitas Sistem / Non-Fungsional (Kuantitatif)

- **Q5: Kebutuhan Sistem Arsip Jangka Panjang yang Terorganisir (Rata-rata Skor: 4,29/5):**
 - **Insight Validasi:** Dokumen seperti transkrip, sertifikat SKEK, dan KHS adalah investasi jangka panjang mahasiswa yang dipakai hingga yudisium bahkan melamar pekerjaan. Sistem harus didesain untuk *storage* berkelanjutan, bukan sekadar penyimpanan sementara.
- **Q6: Sistem untuk Mempercepat Pencarian (Rata-rata Skor: 4,35/5):**
 - **Insight Validasi:** Efisiensi waktu adalah prioritas utama pengguna. Kecepatan menemukan *file* mendadak (seperti saat akan mendaftar beasiswa atau sidang) menjadi indikator kesuksesan aplikasi ini.
- **Q7: Kemampuan Klasifikasi Dokumen Otomatis (Rata-rata Skor: 4,16/5):**
 - **Insight Validasi:** Fitur *Machine Learning* yang ditawarkan untuk mengenali jenis teks dokumen terbukti dibutuhkan mahasiswa. Pengguna ingin dibebaskan dari beban menyortir dokumen ke dalam *folder* secara manual.
- **Q8: Kemudahan Penggunaan Tanpa Pelatihan/UI-UX (Rata-rata Skor: 4,35/5):**
 - **Insight Validasi:** Aplikasi ini menyasar mahasiswa dari berbagai tingkat literasi digital dasar hingga menengah. Antarmuka sistem harus dirancang se-intuitif mungkin (misalnya mirip dengan antarmuka Google Drive yang sudah familiar) agar pengguna bisa langsung beradaptasi.
- **Q9: Keamanan dan Privasi Tinggi (Rata-rata Skor: 4,67/5 - TERTINGGI):**

- **Insight Validasi:** Ini merupakan validasi terkuat. Kebutuhan akan keamanan menempati peringkat tertinggi secara absolut. Sejalan dengan hasil wawancara (kekhawatiran kebocoran KTP/Ijazah seperti kasus BPJS), sistem **wajib** mengimplementasikan fitur enkripsi tingkat tinggi dan proteksi data agar privasi terjamin mutlak.
- **Q10: Performa / Waktu Pemrosesan Cepat (Rata-rata Skor: 4,58/5):**
 - **Insight Validasi:** Mahasiswa sangat tidak toleran terhadap aplikasi yang lambat (*lemot*). Karena aplikasi ini menggunakan *Machine Learning* untuk mengekstraksi teks (*text classification*), pihak pengembang harus memastikan *server* kuat dan pemrosesan algoritma di latar belakang berjalan sangat cepat.

C. Analisis Kebutuhan Fitur Fungsional (Kuantitatif)

- **Q11: Fitur Registrasi Akun Pribadi (Rata-rata Skor: 3,93/5 - TERENDAH):**
 - **Insight Validasi:** Meski masih tergolong positif, nilai ini adalah yang terendah di kuesioner. Hal ini memvalidasi masukan bahwa mahasiswa cenderung malas membuat akun baru. Mengintegrasikan sistem ini langsung dengan *Single Sign-On* (SSO) dari ekosistem kampus "Sinau" akan jauh lebih disukai daripada fitur registrasi mandiri.
 - **Q12: Fitur Login Aman (Rata-rata Skor: 4,38/5):**
 - **Insight Validasi:** Terlepas dari cara mendaftarnya, proses autentikasi harus aman untuk mencegah peretasan akun.
 - **Q13 & Q14: Fitur Unggah & Penyimpanan Otomatis by Kategori (Rata-rata Skor: 4,12 & 4,29/5):**
 - **Insight Validasi:** Ini adalah inti fungsionalitas sistem. Pengguna ingin proses yang mulus: cukup unggah *file*, dan biarkan sistem (AI) yang bekerja menyortir apakah itu masuk ke *folder* Sertifikat SKEK, Transkrip, atau Dokumen Administrasi.
 - **Q15, Q16, Q17: Fitur Kelola Dokumen - Lihat, Ubah, Hapus (Rata-rata Skor: 4,35; 4,32; 4,19/5):**
 - **Insight Validasi:** Mahasiswa membutuhkan kontrol penuh atas arsip mereka (sistem CRUD). Fitur ubah dan hapus memvalidasi kebutuhan pengguna untuk melakukan *decluttering* (membersihkan *file* sampah) atau merevisi *file* yang salah dikategorikan oleh mesin AI.
 - **Q18: Fitur Pencarian Berbasis Kata Kunci (Rata-rata Skor: 4,41/5):**
 - **Insight Validasi:** Fitur pencarian mendapat atensi kebutuhan yang luar biasa tinggi dari segi fungsional. Ini selaras dengan masalah di Q1; fungsi *search bar* yang canggih layaknya mesin pencari sangat krusial agar pengguna tidak perlu mencari *file* secara hierarkis.
 - **Q19: Fitur Dashboard Statistik (Rata-rata Skor: 4,09/5):**
 - **Insight Validasi:** Mahasiswa membutuhkan ringkasan visual (seperti grafik/angka). Misalnya, informasi visual yang menampilkan "Anda sudah mengumpulkan 15 sertifikat SKEK" agar pemantauan kelengkapan syarat kelulusan/yudisium lebih mudah diukur.
-

Kesimpulan Analisis

Berdasarkan triangulasi data dari kuesioner dan wawancara, dapat disimpulkan bahwa **pengembangan Aplikasi Arsip Pintar sangat mendesak (urgen) dan memiliki pijakan validasi pasar yang sangat kuat dari sisi pengguna (mahasiswa)**. Mahasiswa saat ini terjebak pada metode penyimpanan konvensional yang berantakan dan proses pencarian yang menguras waktu serta bergantung pada pihak lain.

Untuk memastikan aplikasi ini sukses dan diadopsi dengan baik oleh pengguna, tim pengembang perangkat lunak (*developer*) **harus memprioritaskan tiga hal utama secara berurutan:**

1. **Keamanan Level Tertinggi (Skor 4,67/5):** Membangun enkripsi data dan perlindungan privasi harus menjadi fondasi nomor satu, sebelum membangun fitur *machine learning*.
2. **Kecepatan dan Pencarian Akurat (Skor 4,58 & 4,41/5):** Nilai jual sistem ini ada pada fitur *search bar* berbasis kata kunci serta pemrosesan sistem yang kilat agar dokumen bisa ditemukan dalam hitungan detik.
3. **Kemudahan (Frictionless UX):** Mahasiswa membutuhkan sistem penyimpanan otomatis yang mengelompokkan arsip (skor 4,29/5) tanpa mengharuskan mereka belajar cara memakai aplikasinya (skor 4,35/5). Pembuatan *login* juga sebaiknya memanfaatkan integrasi "Sinau" kampus agar meminimalisir hambatan di awal penggunaan pendaftaran (tercermin dari rendahnya skor registrasi manual di angka 3,93/5).

Rancangan Deployment

Untuk arsitektur sistem klasifikasi dan pengarsipan dokumen digital berbasis ML yang sedang Anda rancang bersama tim, skrip *compose* lokal tersebut harus dirombak mengikuti aturan main Docker Swarm.

Di lingkungan Swarm, konsep **Port** dan **Network** berubah drastis:

1. **Dilarang memakai ports: "xxxx:xxxx"**: Mengekspos port secara fisik akan memicu bentrok antar-kelompok. Jalur masuk publik (Frontend, Jupyter, MinIO Console) sepenuhnya diatur lewat **Label Traefik**.
2. **Database & API Tertutup**: Layanan backend (PostgreSQL, MinIO API) tidak perlu diekspos ke publik. Aplikasi Vue/Nginx dan Jupyter Anda bisa langsung memanggilnya secara internal menggunakan nama *service*-nya (misal: `postgres_db:5432`).
3. **Volume Terpusat**: Karena Anda menyimpan *stateful data* (database, storage fisik, dan model Scipy), kontainer-kontainer tersebut wajib "diikat" di server Master agar tidak kehilangan akses ke folder *mount* saat Swarm melakukan *load balancing*.

Berikut adalah skrip Swarm mutakhir yang sudah dilengkapi Nginx (untuk Vue.js) dan sistem *routing* Traefik:

YAML

```
version: '3.8'
```

```
networks:
```

```
  traefik-public:
```

```
    external: true
```

```
volumes:
```

```
  # Gunakan named volumes untuk Swarm (disimpan di /var/lib/docker/volumes/)
```

```
  postgres_data:
```

```
  minio_data:
```

```
  jupyter_notebooks:
```

```
  ml_models:
```

```
services:
```

```
  # 1. Database Metadata (INTERNAL SAJA)
```

```
  postgres_db:
```

```
    image: postgres:15
```

```
    environment:
```

```
      POSTGRES_USER: admin_sad
```

```
      POSTGRES_PASSWORD: password_db
```

```
      POSTGRES_DB: arsip_db
```

```
    volumes:
```

```
      - postgres_data:/var/lib/postgresql/data
```

```
    networks:
```

```
      - traefik-public
```

```
    deploy:
```

placement:

constraints:

- node.role == manager # Wajib di master agar data aman

2. Storage File Fisik (INTERNAL & PUBLIC CONSOLE)

minio_storage:

image: minio/minio

command: server /data --console-address ":9001"

environment:

MINIO_ROOT_USER: admin_minio

MINIO_ROOT_PASSWORD: password_minio

volumes:

- minio_data:/data

Jika ingin pakai TrueNAS, pastikan path di-mount di Master OS:

- /mnt/staging_sad22:/data

networks:

- traefik-public

deploy:

placement:

constraints:

- node.role == manager

labels:

Buka akses Console MinIO ke publik (minio-k1...)

- "traefik.enable=true"

- "traefik.http.routers.minio-k1.rule=Host(`minio-k1-praktikum.61262.online`)"

- "traefik.http.routers.minio-k1.entrypoints=web"

- "traefik.http.services.minio-k1.loadbalancer.server.port=9001"

- "traefik.swarm.network=traefik-public"

3. Environment Machine Learning & EDA (PUBLIC)

jupyter_lab:

image: jupyter/scipy-notebook:latest

environment:

JUPYTER_ENABLE_LAB: "yes"

command: start-notebook.sh --NotebookApp.token="" # Akses tanpa password

volumes:

- jupyter_notebooks:/home/jovyan/work

- ml_models:/home/jovyan/ml_model

networks:

- traefik-public

deploy:

placement:

constraints:

- node.role == manager

labels:

Buka akses Jupyter ke publik (jupyter-k1...)

- "traefik.enable=true"

- "traefik.http.routers.jupyter-k1.rule=Host(`jupyter-k1-praktikum.61262.online`)"

- "traefik.http.routers.jupyter-k1.entrypoints=web"
- "traefik.http.services.jupyter-k1.loadbalancer.server.port=8888"
- "traefik.swarm.network=traefik-public"

4. Frontend Vue.js + Nginx (PUBLIC)

frontend_vue:

image: nginx:alpine # Nanti diganti dengan image Docker hasil build Vue JS kalian

networks:

- traefik-public

deploy:

labels:

Buka akses web utama ke publik (k1...)

- "traefik.enable=true"
- "traefik.http.routers.web-k1.rule=Host(`k1-praktikum.61262.online`)"
- "traefik.http.routers.web-k1.entrypoints=web"
- "traefik.http.services.web-k1.loadbalancer.server.port=80"
- "traefik.swarm.network=traefik-public"

version: '3.8'

networks:

traefik-public:

external: true

volumes:

postgres_data:

minio_data:

jupyter_notebooks:

ml_models:

cloudbeaver_data:

services:

1. Database Metadata

postgres_db:

image: postgres:15

environment:

POSTGRES_USER: \${POSTGRES_USER}

POSTGRES_PASSWORD: \${POSTGRES_PASSWORD}

POSTGRES_DB: \${POSTGRES_DB}

command: -p 5433

volumes:

- postgres_data:/var/lib/postgresql/data

networks:

- traefik-public

deploy:

placement:

constraints:

- node.role == manager

2. Storage File Fisik

minio_storage:

image: minio/minio

environment:

MINIO_ROOT_USER: \${MINIO_ROOT_USER}

MINIO_ROOT_PASSWORD: \${MINIO_ROOT_PASSWORD}

command: server /data --address ":9005" --console-address ":9006"

volumes:

- minio_data:/data

networks:

- traefik-public

deploy:

placement:

constraints:

- node.role == manager

labels:

- "traefik.enable=true"

- "traefik.http.routers.minio-k1.rule=Host(`minio-k1-praktikum.61262.online`)"

- "traefik.http.routers.minio-k1.entrypoints=web"

- "traefik.http.services.minio-k1.loadbalancer.server.port=9006"

- "traefik.swarm.network=traefik-public"

3. Environment Machine Learning & EDA

jupyter_lab:

image: jupyter/scipy-notebook:latest

environment:

JUPYTER_ENABLE_LAB: \${JUPYTER_ENABLE_LAB}

command: start-notebook.sh --port=8889 --NotebookApp.token=""

volumes:

- jupyter_notebooks:/home/jovyan/work

- ml_models:/home/jovyan/ml_model

networks:

- traefik-public

deploy:

placement:

constraints:

- node.role == manager

labels:

- "traefik.enable=true"

- "traefik.http.routers.jupyter-k1.rule=Host(`jupyter-k1-praktikum.61262.online`)"

- "traefik.http.routers.jupyter-k1.entrypoints=web"

- "traefik.http.services.jupyter-k1.loadbalancer.server.port=8889"

- "traefik.swarm.network=traefik-public"

4. Frontend Vue.js + Nginx

frontend_vue:

image: nginx:alpine

networks:

- traefik-public

deploy:

labels:

- "traefik.enable=true"
- "traefik.http.routers.web-k1.rule=Host(`k1-praktikum.61262.online`)"
- "traefik.http.routers.web-k1.entrypoints=web"
- "traefik.http.services.web-k1.loadbalancer.server.port=80"
- "traefik.swarm.network=traefik-public"

Kredensial Database

POSTGRES_USER=admin_sad

POSTGRES_PASSWORD=password_db

POSTGRES_DB=arsip_db

Kredensial MinIO

MINIO_ROOT_USER=admin_minio

MINIO_ROOT_PASSWORD=password_minio

Konfigurasi Jupyter

JUPYTER_ENABLE_LAB=yes

5. Database Manager Berbasis Web (CloudBeaver)

cloudbeaver:

image: dbeaver/cloudbeaver:latest

volumes:

- cloudbeaver_data:/opt/cloudbeaver/workspace

networks:

- traefik-public

deploy:

placement:

constraints:

- node.role == manager

labels:

- "traefik.enable=true"

Buka akses CloudBeaver ke publik (db-k1...)

- "traefik.http.routers.db-k1.rule=Host(`db-k1-praktikum.61262.online`)"

- "traefik.http.routers.db-k1.entrypoints=web"

- "traefik.http.services.db-k1.loadbalancer.server.port=8978" # Port bawaan

CloudBeaver

- "traefik.swarm.network=traefik-public"

Cara Komunikasi Internal (Penting untuk Backend & ML):

Karena port fisik dihapus, kode aplikasi Anda (Vue/Jupyter) harus memanggil layanan lain menggunakan DNS internal Swarm:

- Koneksi ke Postgres:
`postgres://admin_sad:password_db@postgres_db:5432/arsip_db`
- Koneksi API ke MinIO: `http://minio_storage:9000`

Tab 17

Skenario pengujian (testing) untuk *Access* dan *Refresh Token* sangat penting untuk memastikan fitur **Silent Refresh** kamu berjalan mulus di "Arsip Pintar". Tanpa pengujian yang tepat, kamu tidak akan tahu apakah user benar-benar mendapatkan token baru atau sebenarnya mereka sedang "dipaksa" login berulang kali.

Berikut adalah skenario "Tempur" untuk menguji sistem kamu:

1. Persiapan: Percepat Waktu Kedaluwarsa

Jangan menunggu 15 menit hanya untuk satu kali tes. Ubah konfigurasi di backend (`.env` atau file *config*) agar token mati sangat cepat.

- **ACCESS_TOKEN_EXPIRE_MINUTES:** `1` (1 menit).
 - **REFRESH_TOKEN_EXPIRE_DAYS:** `2` (atau biarkan default, yang penting lebih lama dari access token).
-

2. Skenario Tes: Silent Refresh Berhasil

Tujuannya untuk memastikan saat Access Token mati, Axios diam-diam mengambil yang baru tanpa mengganggu user.

1. **Langkah:** Buka aplikasi, lalu **Login**.
 2. **Langkah:** Buka **Inspect Element (F12)** -> Tab **Network**.
 3. **Langkah:** Tunggu selama **1 menit 5 detik** (sampai Access Token dipastikan mati).
 4. **Langkah:** Klik menu yang memanggil API (misal: buka halaman "User Management" atau "Repository").
 5. **Hasil yang Diharapkan:**
 - Kamu akan melihat request pertama ke API (misal: `/users`) berstatus **401 Unauthorized**.
 - Tepat setelah itu, ada request otomatis ke `/refresh` berstatus **200 OK**.
 - Terakhir, request `/users` tadi dipanggil ulang otomatis dan berstatus **200 OK**.
 - **User sama sekali tidak ditendang ke halaman login.**
-

3. Skenario Tes: Refresh Token Expired/Revoked

Tujuannya memastikan jika sesi benar-benar habis, user wajib login ulang demi keamanan.

1. **Langkah:** Login seperti biasa.
2. **Langkah:** Buka **Inspect Element** -> Tab **Application** -> **Cookies**.
3. **Langkah:** Cari cookie `refresh_token` lalu **Hapus** secara manual (simulasi token hilang/expired).
4. **Langkah:** Coba akses menu atau lakukan tindakan (seperti hapus data).

5. Hasil yang Diharapkan:

- Request ke API menghasilkan **401**.
 - Request ke `/refresh` juga menghasilkan **401** (karena cookie sudah hilang).
 - Axios Interceptor menangkap kegagalan ini di blok `catch`.
 - **User otomatis diarahkan (redirect) ke halaman `/login`.**
-

4. Skenario Tes: Multi-Tab (Sesi Konsisten)

Memastikan status login tetap terjaga di tab berbeda.

1. **Langkah:** Login di Tab A.
 2. **Langkah:** Buka Tab B dengan URL aplikasi yang sama.
 3. **Hasil yang Diharapkan:** User harus sudah dalam keadaan login di Tab B (karena `localStorage` atau `sessionStorage` berbagi data, dan cookie `refresh_token` dikirim secara global untuk domain tersebut).
-

Cara Membaca Hasil di Network Tab

Untuk mempermudah kamu saat debugging, perhatikan kolom **Initiator** di tab Network:

- Request pertama berasal dari komponen Vue kamu (misal: `UserManagement.vue`).
 - Request `/refresh` harusnya berasal dari `api/index.ts` (Interceptor).
 - Request ulang (retry) juga harusnya dipicu oleh `api/index.ts`.
-

Tips Debugging Tambahan

Tambahkan `console.log` di Interceptor kamu untuk mempermudah pemantauan:

TypeScript

// Di api/index.ts

```
if (error.response && error.response.status === 401) {  
  console.log("🚩 Mendeteksi 401, mencoba menyelamatkan sesi...");  
  // ... logika refresh ...  
}
```

Jika saat pengujian kamu melihat request `/refresh` dipanggil berulang-ulang tanpa henti, segera cek poin **"Infinite Loop"** yang kita bahas sebelumnya di bagian `originalRequest.url.includes('/login')`.